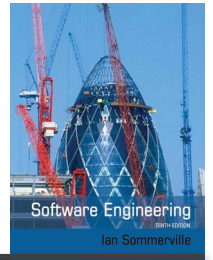


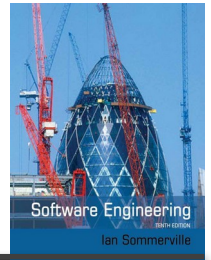
Chapter 4 – Requirements Engineering

Topics covered



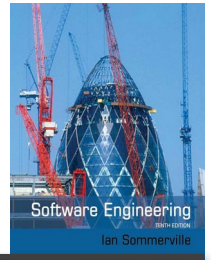
- ✧ Functional and non-functional requirements
- ✧ Requirements engineering processes
- ✧ Requirements elicitation
- ✧ Requirements specification
- ✧ Requirements validation

Requirements engineering



- ✧ The process of establishing the services that a customer requires from a system and the constraints under which it operates and is developed.
- ✧ The system requirements are the descriptions of the system services and constraints that are generated during the requirements engineering process.

Types of requirement



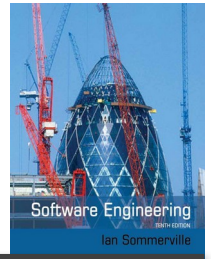
✧ User requirements

- Statements in natural language plus diagrams of **the services the system provides and its operational constraints**. Written for customers.

✧ System requirements

- A structured document setting out **detailed descriptions of the system's functions**, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

User and system requirements



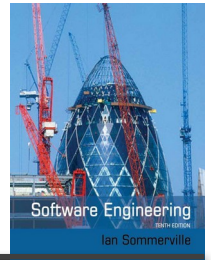
User requirements definition

- 1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc) separate reports shall be created for each dose unit.
- 1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

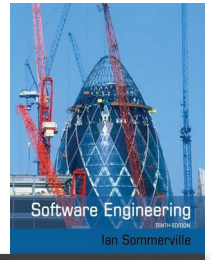
System stakeholders



- ✧ **Any person or organization who is affected by the system in some way** and so who has a legitimate interest
- ✧ Stakeholder types
 - End users
 - System managers
 - System owners
 - External stakeholders

Functional and non-functional requirements

Functional and non-functional requirements



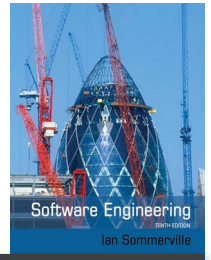
✧ Functional requirements

- **Statements of services the system should provide**, how the system should **react to particular inputs** and how the system should behave in particular situations.
- May state what the system should not do.

✧ Non-functional requirements

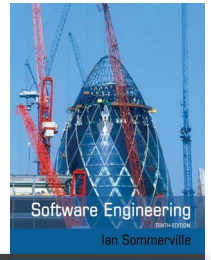
- Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
- Often apply to the system as a whole rather than individual features or services.

Functional requirements



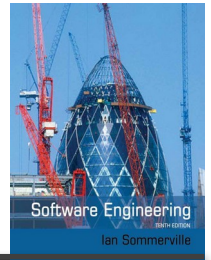
- ✧ **Describe** functionality or system **services**.
- ✧ Depend on the type of software, expected users and the type of system where the software is used.
- ✧ Functional user requirements may be **high-level statements of what the system should do**.
- ✧ Functional system requirements should describe **the system services in detail**.

Mentcare system: functional requirements



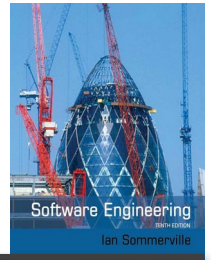
- ✧ A user shall be able to search the appointments lists for all clinics.
- ✧ The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
- ✧ Each staff member using the system shall be uniquely identified by his or her 8-digit employee number.

Requirements imprecision



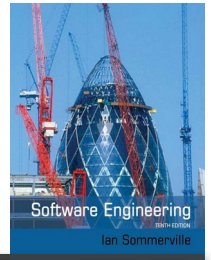
- ✧ Problems arise when functional requirements are not precisely stated.
- ✧ Ambiguous requirements may be interpreted in different ways by developers and users.
- ✧ Consider the term 'search' in requirement 1
 - User intention – search for a patient name across all appointments in all clinics;
 - Developer interpretation – search for a patient name in an individual clinic. User chooses clinic then search.

Requirements completeness and consistency



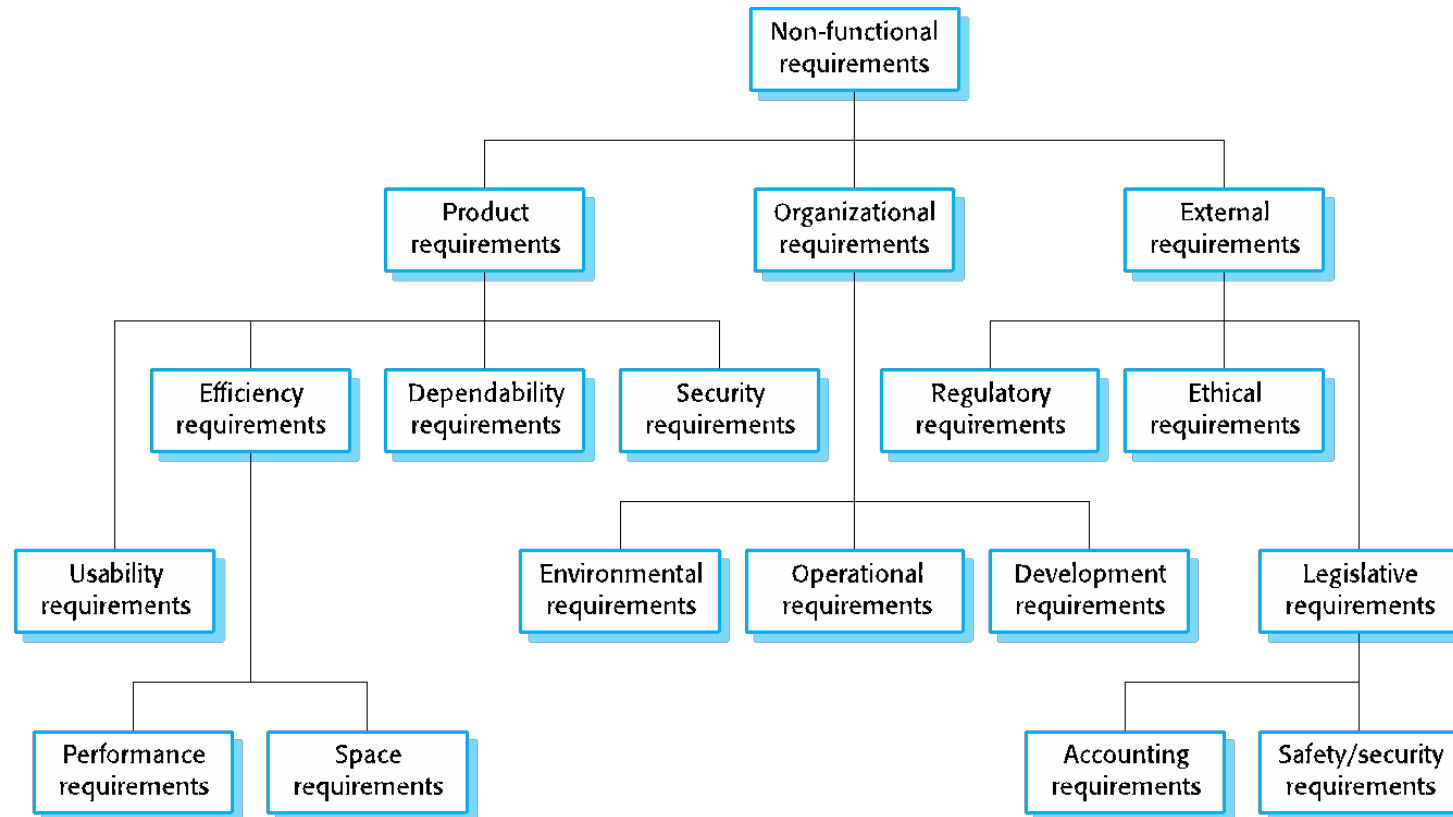
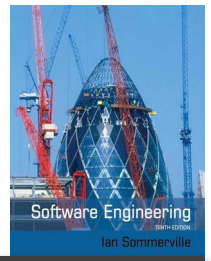
- ✧ In principle, requirements should be both **complete and consistent**.
- ✧ Complete
 - They should include descriptions of all facilities required.
- ✧ Consistent
 - There should be no conflicts or contradictions in the descriptions of the system facilities.
- ✧ In practice, because of system and environmental complexity, it is impossible to produce a complete and consistent requirements document.

Non-functional requirements

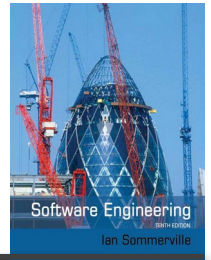


- ✧ These define **system properties and constraints** e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.
- ✧ Process requirements may also be specified mandating a particular IDE, programming language or development method.
- ✧ **Non-functional requirements** may be more critical than functional requirements. If these are not met, the system may be useless.

Types of nonfunctional requirement

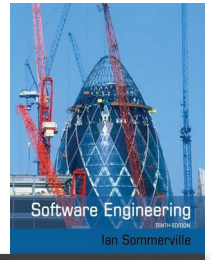


Non-functional requirements implementation



- ✧ Non-functional requirements may **affect the overall architecture of a system rather than the individual components.**
 - For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
- ✧ A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
 - It may also generate requirements that restrict existing requirements.

Non-functional classifications



✧ Product requirements

- Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.

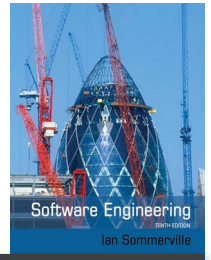
✧ Organisational requirements

- Requirements which are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.

✧ External requirements

- Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Examples of nonfunctional requirements in the Mentcare system



Product requirement

The Mentcare system shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.

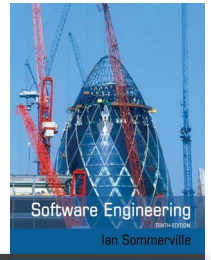
Organizational requirement

Users of the Mentcare system shall authenticate themselves using their health authority identity card.

External requirement

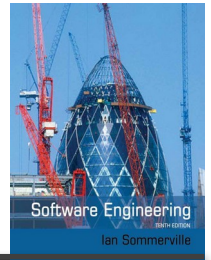
The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

Goals and requirements



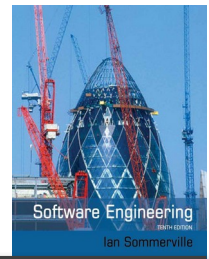
- ✧ Non-functional requirements may be very difficult to state precisely and imprecise requirements may be difficult to verify.
- ✧ Goal
 - A general intention of the user such as ease of use.
- ✧ Verifiable non-functional requirement
 - A statement using some measure that can be objectively tested.
- ✧ Goals are helpful to developers as they convey the intentions of the system users.

Usability requirements



- ✧ The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized. (Goal)
- ✧ Medical staff shall be able to use all the system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use. (Testable non-functional requirement)

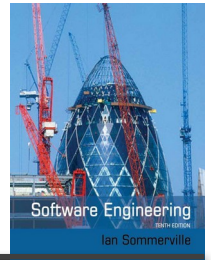
Metrics for specifying nonfunctional requirements



Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Requirements engineering processes

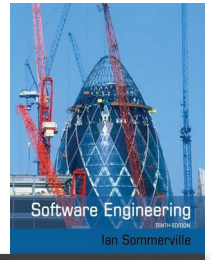
Requirements engineering processes



- ✧ The processes used for RE vary widely depending on the application domain, the **people involved** and the organisation **developing the requirements**.
- ✧ However, there are a number of generic activities common to all processes
 - **Requirements elicitation;**
 - **Requirements analysis;**
 - **Requirements validation;**
 - **Requirements management.**
- ✧ In practice, RE is an iterative activity in which these processes are interleaved.

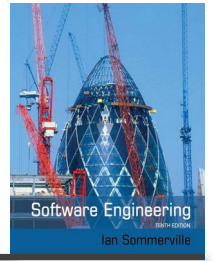
Requirements elicitation

Requirements elicitation and analysis



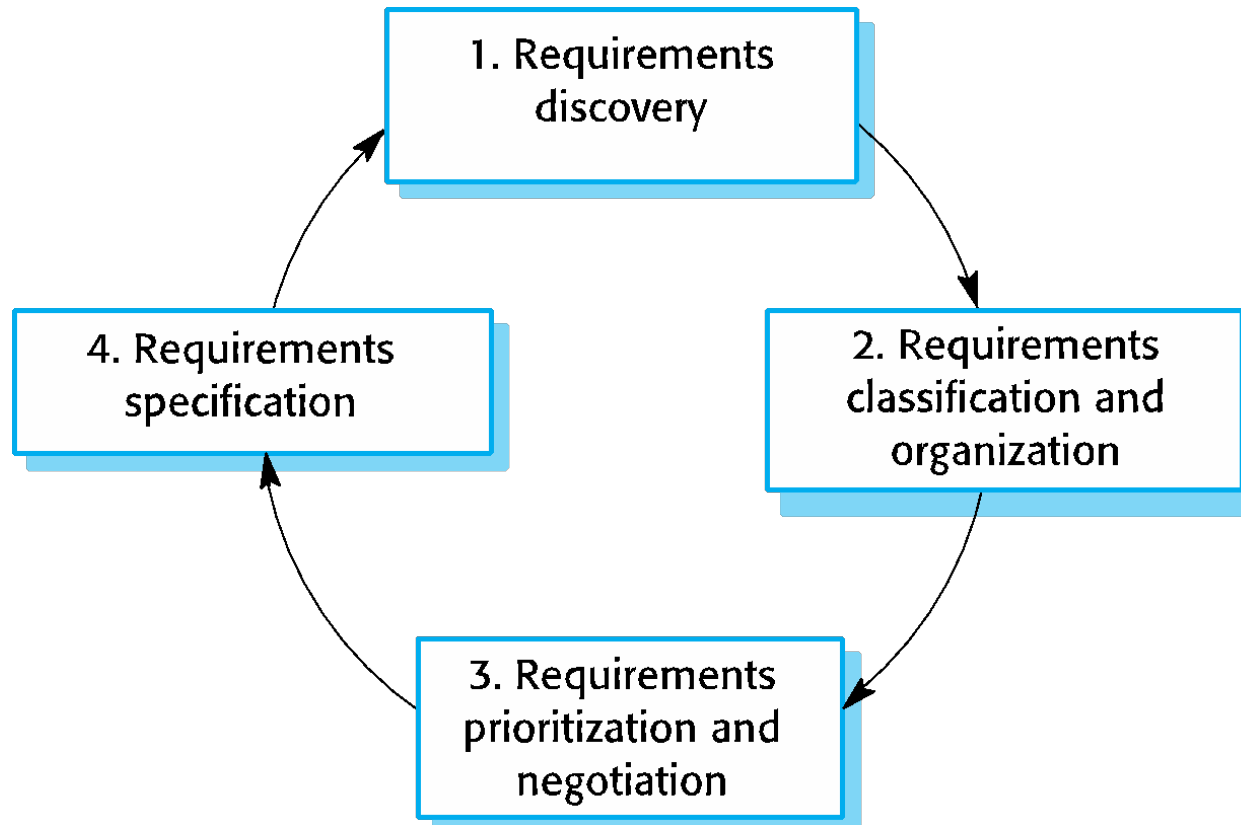
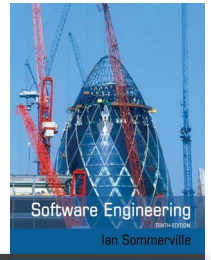
- ✧ Sometimes called requirements **elicitation** or requirements **discovery**.
- ✧ **Involves technical staff** working with customers to find out about the application domain, **the services that the system should provide** and **the system's operational constraints**.
- ✧ May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called ***stakeholders***.

Problems of requirements elicitation

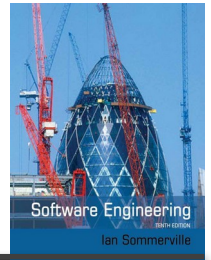


- ✧ Stakeholders **don't know what they really want.**
- ✧ Stakeholders express requirements in their **own terms.**
- ✧ Different stakeholders may have **conflicting requirements.**
- ✧ Organisational and political factors may influence the system requirements.
- ✧ The requirements change during the analysis process. **New stakeholders** may emerge and the business environment may change.

The requirements elicitation and analysis process



Process activities



✧ Requirements discovery

- **Interacting with stakeholders** to discover their requirements. Domain requirements are also discovered at this stage.

✧ Requirements classification and organisation

- **Groups related requirements** and **organises** them into coherent clusters.

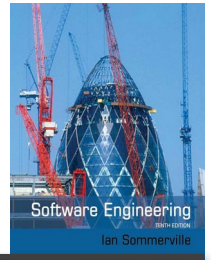
✧ Prioritisation and negotiation

- **Prioritising** requirements and resolving requirements **conflicts**.

✧ Requirements specification

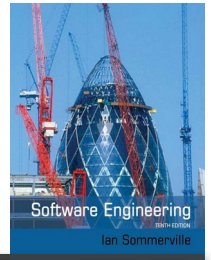
- **Requirements are documented** and input into the next round of the spiral.

Interviewing



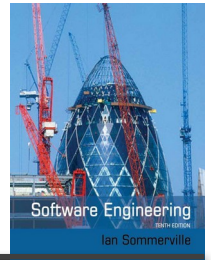
- ✧ **Formal or informal interviews** with stakeholders are part of most RE processes.
- ✧ Types of interview
 - **Closed interviews** based on pre-determined list of questions
 - **Open interviews** where various issues are explored with stakeholders.
- ✧ Effective interviewing
 - Be **open-minded**, avoid pre-conceived ideas about the requirements and are willing to listen to stakeholders.
 - Prompt the interviewee to get discussions going using a springboard question, a requirements proposal, or by working together on a prototype system.

Problems with interviews



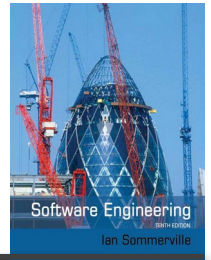
- ✧ Application specialists may use language to describe their work that **isn't easy for the requirements engineer to understand.**
- ✧ Interviews are **not good for understanding domain requirements**
 - Requirements engineers cannot understand specific domain terminology;
 - Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.

Ethnography (skip)



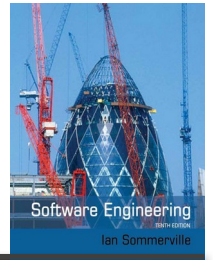
- ✧ A social scientist spends a considerable time observing and analysing how people actually work.
- ✧ People do not have to explain or articulate their work.
- ✧ Social and organisational factors of importance may be observed.
- ✧ Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

Stories and scenarios



- ✧ Scenarios and user stories are real-life examples of how a system can be used.
- ✧ **Stories and scenarios are a description of how a system may be used for a particular task.**
- ✧ Because they are based on a practical situation, stakeholders can relate to them and can comment on their situation with respect to the story.

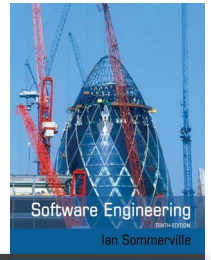
Scenarios



- ✧ A structured form of user story
- ✧ Scenarios should include
 - A description of the starting situation;
 - A description of the normal flow of events;
 - A description of what can go wrong;
 - Information about other concurrent activities;
 - A description of the state when the scenario finishes.

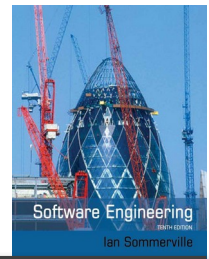
Requirements specification

Requirements specification



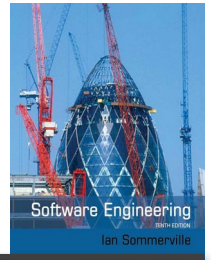
- ✧ **The process of writing down the user and system requirements in a requirements document.**
- ✧ **User requirements have to be understandable by end-users and customers who do not have a technical background.**
- ✧ **System requirements are more detailed requirements and may include more technical information.**
- ✧ **The requirements may be part of a contract for the system development**
 - It is therefore important that these are as complete as possible.

Ways of writing a system requirements specification



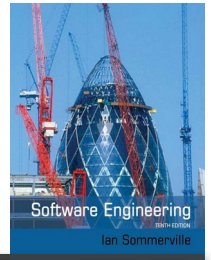
Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages (skip)	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications (chapter 22)	These notations are based on mathematical concepts such as finite-state machines or sets . Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Natural language specification



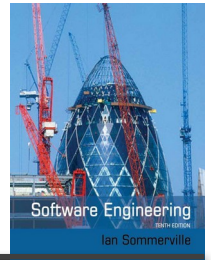
- ✧ Requirements are written **as natural language sentences supplemented by diagrams and tables.**
- ✧ Used for writing requirements because it is expressive, intuitive and universal. This means that the requirements can be understood by users and customers.

Guidelines for writing requirements



- ✧ Invent a **standard format** and use it for all requirements.
- ✧ **Use language in a consistent way.** Use **shall** for **mandatory requirements**, **should** for **desirable requirements**.
- ✧ Use text highlighting to identify key parts of the requirement.
- ✧ Avoid the use of computer jargon.
- ✧ Include an explanation (rationale) of why a requirement is necessary.

Problems with natural language



✧ Lack of clarity

- Precision is difficult without making the document difficult to read.

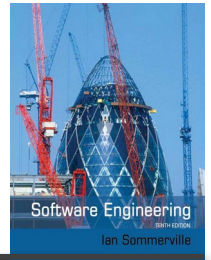
✧ Requirements confusion

- Functional and non-functional requirements tend to be mixed-up.

✧ Requirements amalgamation

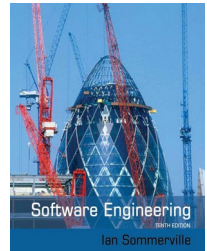
- Several different requirements may be expressed together.

Structured specifications



- ✧ An approach to writing requirements where the **freedom of the requirements writer is limited** and **requirements are written in a standard way**.
- ✧ This works well for some types of requirements e.g. requirements for embedded control system but is sometimes too rigid for writing business system requirements.

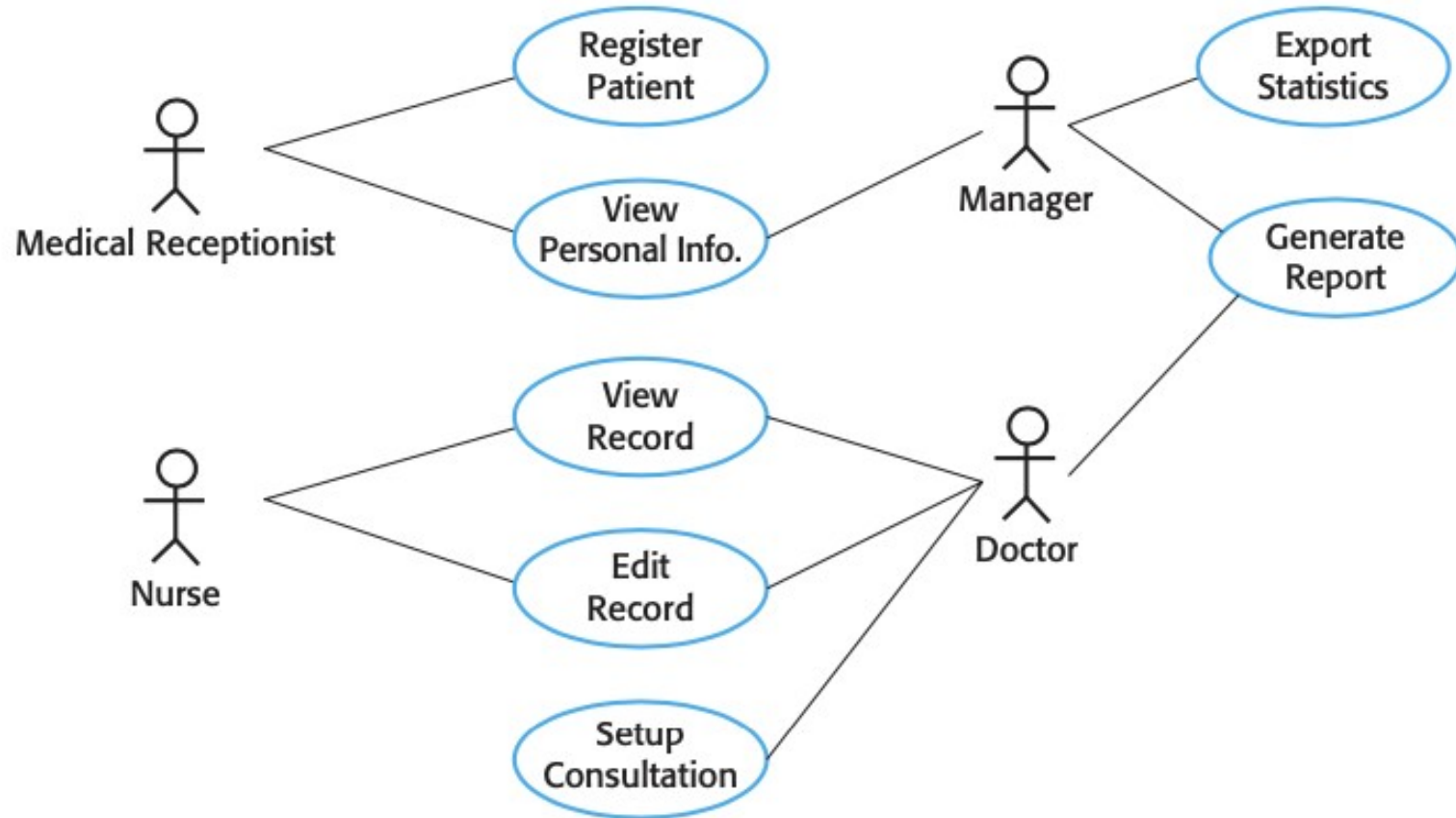
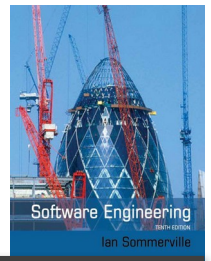
Form-based specifications



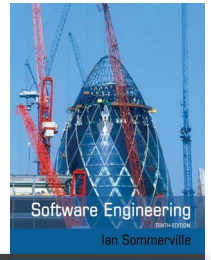
Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.
Requirements	Two previous readings so that the rate of change of sugar level can be computed.
Pre-condition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Post-condition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

Graphical notations (Ex. Use case diagram)

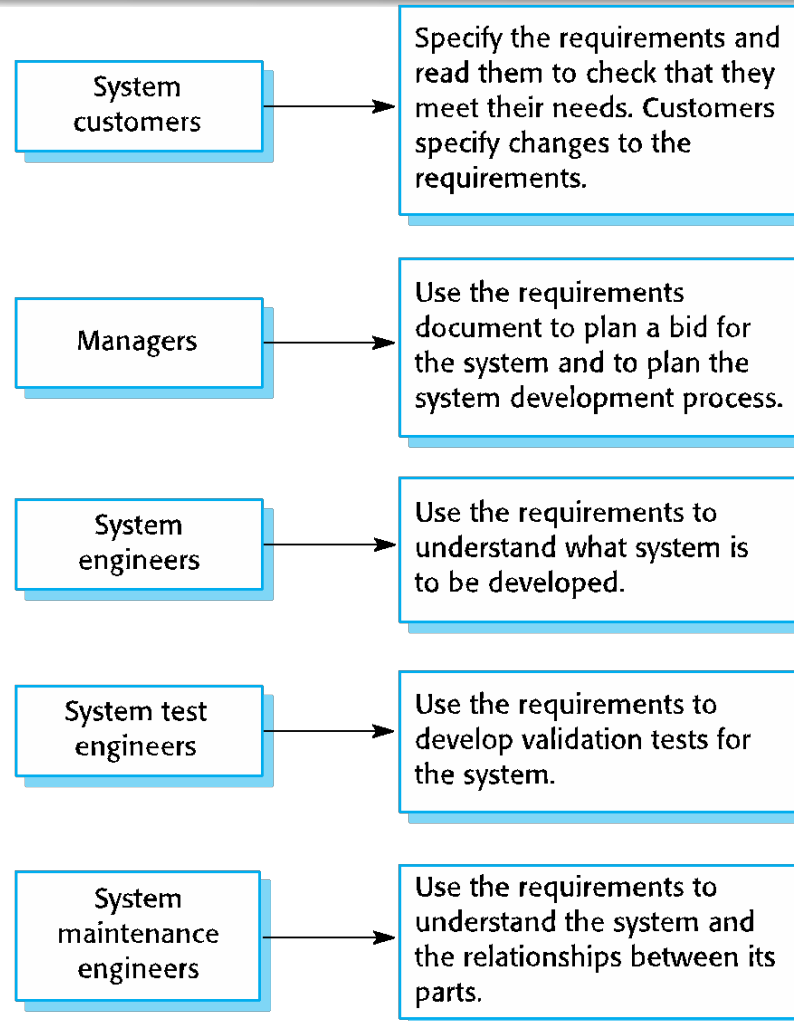
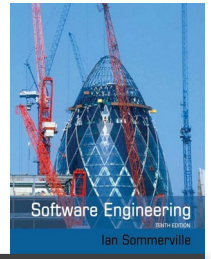


The software requirements document



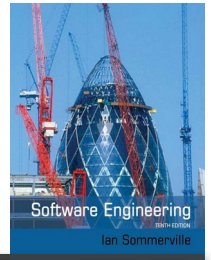
- ✧ The software requirements document is the official statement of what is required of the system developers.
- ✧ Should include both a definition of user requirements and a specification of the system requirements.
- ✧ It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

Users of a requirements document



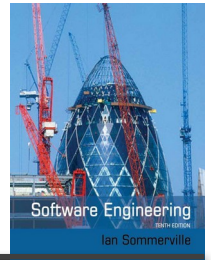
Requirements validation

Requirements validation



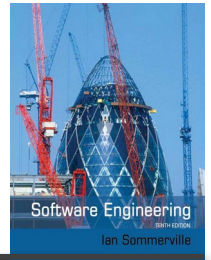
- ✧ Concerned with demonstrating that the requirements define the system that the customer really wants.
- ✧ Requirements error costs are high so validation is very important
 - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking



- ✧ **Validity.** Does the system provide the functions which best support the customer's needs?
- ✧ **Consistency.** Are there any requirements conflicts?
- ✧ **Completeness.** Are all functions required by the customer included?
- ✧ **Realism.** Can the requirements be implemented given available budget and technology
- ✧ **Verifiability.** Can the requirements be checked?

Requirements validation techniques



✧ Requirements reviews

- Systematic manual analysis of the requirements.

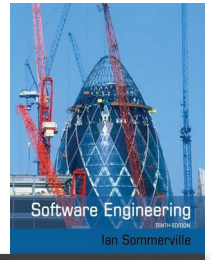
✧ Prototyping

- Using an executable model of the system to check requirements.

✧ Test-case generation

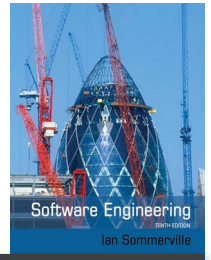
- Developing tests for requirements to check testability.

Requirements reviews



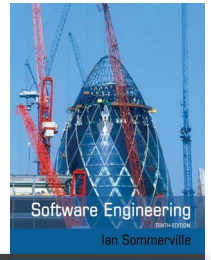
- ✧ Regular reviews should be held while the requirements definition is being formulated.
- ✧ Both **client** and **contractor** staff should be involved in reviews.
- ✧ Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Key points



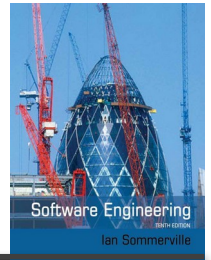
- ✧ Requirements for a software system set out what the system should do and define constraints on its operation and implementation.
- ✧ Functional requirements are statements of the services that the system must provide or are descriptions of how some computations must be carried out.
- ✧ Non-functional requirements often constrain the system being developed and the development process being used.
- ✧ They often relate to the emergent properties of the system and therefore apply to the system as a whole.

Key points



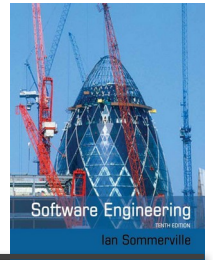
- ✧ The requirements engineering process is an iterative process that includes requirements elicitation, specification and validation.
- ✧ You can use a range of techniques for requirements elicitation including interviews and ethnography. User stories and scenarios may be used to facilitate discussions.

Key points



- ✧ Requirements specification is the process of formally documenting the user and system requirements and creating a software requirements document.
- ✧ The software requirements document is an agreed statement of the system requirements. It should be organized so that both system customers and software developers can use it.

Key points



- ✧ Requirements validation is the process of checking the requirements for validity, consistency, completeness, realism and verifiability.
- ✧ Business, organizational and technical changes inevitably lead to changes to the requirements for a software system. Requirements management is the process of managing and controlling these changes.